

NEXT STEPS: MCU + APP

One repository. Separate firmware and app builds. Complete the device handshake next.

START HERE: secure BLE + implement firmware credential transactions.

The Flutter client is implemented. Firmware status and scans exist; credential transfer is still missing.

READY IN HOST TESTS

15 shared tests pass; measurement formats match.

CLIENT IMPLEMENTED

Flutter setup, change/forget, errors and encryption checks.

FIRMWARE PARTIAL

BEGIN, credential Data, COMMIT, CLEAR and CANCEL missing.

BOARD ACCEPTANCE PENDING

Pairing, NVS, live connection and 1 kHz behavior need proof.

Do these in order

1

Secure BLE + stage credential transactions

Device rejects unsecured writes; fragments stage safely without changing NVS.

FIRMWARE

2

Save, connect and forget without reboot

COMMIT connects; CLEAR leaves an unprovisioned device able to scan and configure again.

FIRMWARE

3

Publish live status, errors and reconnects

Phone shows success/failure and supports retry/edit; router or BLE loss never stops acquisition.

FIRMWARE + APP

4

Run board + end-to-end acceptance

Persistence, interrupted/malformed/security cases, history and sampling evidence all pass.

JOINT BENCH TEST

5

Review evidence; prepare the later merge

Record checks/builds/board results. Merge only after the user requests it.

TEAM

KEEP FIXED 16 channels, existing measurement UUIDs, 88-byte UDP frames, 80-byte BLE values, and the 1 kHz acquisition path.

Roles are suggested work areas; individual owners and dates have not been assigned.

IMPLEMENTATION MAP

Use the existing protocol and WiFiManager. Build the missing firmware side in this order.

1. Secure link + bounded staging

```
components/ble/ble_manager.cpp
components/ble/include/provisioning_protocol.hpp
components/ble/include/wifi_provisioning.hpp + components/ble/wifi_provisioning.cpp
```

Enforce encrypted links for BEGIN/Data/COMMIT/CLEAR on the device. Configure LE Secure Connections and define authenticated pairing UX. Stage at most 97 bytes (SSID 32, password 63); use matching transaction IDs and 13-byte chunks at MTU 23. Validate identity, constant length, offsets and conflicting duplicates. Expire staging in about 60 seconds; clear sensitive copies on cancel/expiry/disconnect cleanup. Queue flash and radio work off GATT callbacks; invalidate old connection work.

Done when: Bad or interrupted requests leave NVS unchanged. No password logging, crash, or acquisition changes.

2. Apply / clear without reboot

```
components/wifi/include/wifi_manager.hpp
components/wifi/wifi_manager.cpp + components/wifi/wifi_credentials.cpp
```

Keep the existing wifi_cfg NVS namespace and ssid/password keys. saveCredentials() currently persists only: add worker-owned live apply/connect after a valid COMMIT. clearCredentials() currently deinitializes Wi-Fi: restore an UNPROVISIONED station able to scan afterward. Serialize mutations with scanning on one communications worker; keep development seeding off for final firmware.

Done when: Change and forget/reconfigure work in one boot. Saved credentials survive power cycle; cleared ones stay absent.

3. Live state + app handoff

```
Firmware: components/ble/wifi_provisioning.cpp + components/wifi/wifi_manager.cpp
App: app/apps/monitor/lib/{wifi_protocol,wifi_transport,wifi_settings}.dart
```

Publish state, IPv4, error and encryption changes through the NimBLE host event queue. Current firmware notifies only on explicit requests/scan events. Keep a visible failure/timeout while retaining reconnects. Verify app success/failure, retry/edit, change/forget and fresh encrypted-link checks. The BLE layer queries WiFiManager; it must not own another Wi-Fi state machine.

Done when: The phone sees actionable updates after wrong passwords or router loss; BLE and acquisition continue.

BOUNDARY TO PRESERVE

BLE sets router credentials. UDP sends measurements to the laptop; the mobile collector logs and uploads to the API/database. Configure the laptop IP separately in components/udp/include/udp_config.hpp. API/cloud account setup and destination discovery are outside provisioning v1.

ACCEPTANCE + HANDOFF

Run the board cases in section 27 of the specification. Keep a result and evidence for each.

REQUIRED CASE	EXPECTED RESULT / EVIDENCE
Fresh device / MTU 23 scan	UNPROVISIONED offers setup; supported network results reassemble. Keep the app/status trace.
Correct / wrong password	Connects or shows actionable failure; retry works; no reboot or password logs.
Power cycle / change / forget	NVS persists valid credentials; changing works live; forgetting permits setup again in one boot.
Disconnect / cancel / expiry	Partial staging clears safely in about 60 s; prior stored credentials remain unchanged.
Malformed / unsecured input	Firmware rejects invalid or unsecured sensitive writes without crash or partial save.
Router loss / acquisition	Recovery resumes; BLE continues; about 1,000 frames/s and no provisioning-caused queue overflow.
Collector to web history	UDP alone stores zero rows. Collector logs/uploads 16 channels; retries do not duplicate history.

Run these checks before review

HOST CONTRACTS From repository root after app setup:

```
python tools/check_integration.py
```

FULL APP + LOCAL POSTGRES From the app/ folder:

```
dev.cmd test
```

FIRMWARE / ACTUAL BOARD In its ESP-IDF terminal at root:

```
idf.py build
```

```
idf.py -p COM3 flash monitor
```

Use the actual target and serial port; COM3 is a placeholder. Check image/partition and task-stack headroom. Retest the planned ESP32-S3 separately.

RECORD AND REVIEW BEFORE THE LATER MERGE

Record commit, board/phone versions, target, pass/fail and diagnostic evidence. Link firmware/bench results from docs/ and update app/docs/verification.md. Host baseline: 15 tests passed. Hardware/security/NVS/timing remain unverified; --skip-flutter is partial. Merge only when requested.

Separate later work: iOS native BLE adapter/Mac tests; optional USB snapshot wiring; real ADC/calibration and other features outside provisioning v1. The editable NEXT_STEPS.md has full paths, all cases and commands.